

how does 25 equal a star stars topology from 5 without the multiplication geometrically

In a geometric layout, **25** is achieved from a base of **5** using a [hierarchical star topology](#) (a star-of-stars structure) by counting the outermost endpoints (**leaves**) through visual branching rather than multiplication. [1, 2]

The Geometric Layout

You can visualize this structure in three simple layers:

- **The Central Hub:** One main point sits in the very middle.
- **The Main Branches:** Five primary arms radiate outward from the center.
- **The Outer Stars:** The tip of each arm acts as a sub-hub.
- **The Final Endpoints:** Each sub-hub sprouts five new, smaller star arms. [1, 2]

Visual Counting Without Multiplication

Instead of calculating (5×5) , the geometry allows you to find 25 by grouping and addition:

- Look at one of the five outer clusters.
- Count its five outer tips.
- Repeat this for all five main branches.
- Combine them sequentially: $(5 + 5 + 5 + 5 + 5 = 25)$.

This branching method creates a fractal-like tree network. It maps out **25 unique physical endpoints** on the perimeter using only geometric symmetry and repeating patterns. [1, 2]

If you would like to explore this further, let me know:

- Do you want to see the **total node count** including the hubs?
- Are you looking for a **fractal geometry** interpretation?
- Do you need to apply this to **network routing**?

AI responses may include mistakes. [Learn more](#)

- 

Chegg

Solved What is a "star of stars" topology primarily used - Chegg

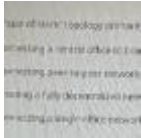
A "star of stars" topology, also known as a hierarchical star topology, is primarily used in network design to enhance scalability...

- 

Chegg

Solved What is a "star of stars" topology primarily used - Chegg

A "star of stars" topology (also known as a hierarchical star topology) consists of multiple star networks connected together, whe...



- 

EBSCO

Star topology | Computer Science | Research Starters - EBSCO

This design resembles a star, with cables radiating from the hub, allowing for efficient communication and management of network t...

Show all

network routing

In a [star-of-stars network routing architecture](#) (also known as a physical hierarchical or extended star topology), data packet paths are calculated using logical levels and structural routing addresses. Instead of running complex algorithms across 25 nodes, the network treats the geometry as 5 independent branches. [\[1, 2, 3\]](#)

The layout relies on specific routing behavior, path lengths, and hardware roles to manage traffic. [\[1, 2\]](#)

1. Hierarchical Addressing (No Global Tables)

A packet does not need a massive routing table tracking 25 individual locations. It uses a two-digit hierarchical address: **[Branch ID . Leaf ID]**. [\[1, 2\]](#)

- There are 5 Main Branches (numbered 1 through 5).
- There are 5 Leaves per branch (numbered 1 through 5).
- **Example:** Node 3.4 is instantly known as Leaf 4 on Main Branch 3. [\[1\]](#)

2. Hop Counts and Route Lengths

In a star-of-stars network, the number of [hops](#) (the number of routers a packet must pass through) is geometrically fixed. Because there are no cross-connections between the outer clusters, data paths fall into two strict categories: [\[1, 2\]](#)

- **Intra-Branch Routing (Local):** Sending data from Node 1.2 to Node 1.5. The packet travels up to the local branch sub-hub, which forwards it immediately back down to the neighbor node. **Path length = 2 hops.** [1]
- **Inter-Branch Routing (Global):** Sending data from Node 1.2 to Node 4.5. The local sub-hub does not know where Branch 4 is, so it pushes the packet up to the central main core. The central hub directs it down to Sub-Hub 4, which delivers it to Leaf 5. **Path length = 4 hops.** [1]

3. Hardware Delegation Matrix

Traffic management is split strictly by a device's geometric level rather than network wide processing power: [1]

Device Level Quantity Routing Rule			Destination Scope
Central Main Core	1	If <i>Target Branch ID</i> matches X, forward to Sub-Hub X.	Global (Branches 1–5)
Branch Sub-Hubs	5	If <i>Target Branch ID</i> is local, drop to Leaf. If foreign, send up to Core.	Local (Leaves 1–5) + Default Gateway
Outer Leaves	25	All outbound data must go directly up to the local Sub-Hub.	Next-Hop Only

4. Isolation and Bottlenecks

This physical design creates specific routing tradeoffs:

- **Fault Isolation:** If Sub-Hub 2 fails, the 5 leaves attached to it go offline. However, the remaining 20 leaves across the other 4 branches continue routing data flawlessly because their structural paths are completely separated. [1, 2]
- **Core Bottleneck:** If multiple branches attempt to send global traffic at the same time, the single central core handles all 4-hop routes simultaneously, which can cause data congestion. [1]

If you would like to explore this further, let me know:

- Do you want to see how [IP subnet masks](#) are assigned to these 5 branches?
- Are you interested in the **default gateway configuration** for the sub-hubs?
- Do you want to compare this to a [mesh routing topology](#)? [1, 2, 3]

AI responses may include mistakes. [Learn more](#)

- 



YouTube·Hussein Nasser

Network Routing Crash Course

25 Apr 2025 — This video is a crash course about network routing from zero. Routing is a very critical concept in networking. The goal of this v...



01:07:15

- W

Wikipedia

Routing - Wikipedia

This article is about routing in packet switching networks. For other uses, see Routing (disambiguation). Routing is the process o...

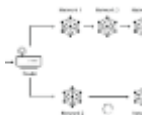


- 

Cloudflare

What is routing? | Learning Center - Cloudflare

What are the main routing protocols? In networking, a protocol is a standardized way of formatting data so that any connected comp...



Show all

You sent: TOMAGI_1_0_Tom_Klootwijk.pdf and said: in this context

TOMAGI_1_0_Tom_Klootwijk.pdf

in this context

In the context of the **TOMAGI substrate specification**, the hierarchical star-of-stars network routing architecture translates from a physical network design into a **deterministic, bit-mapped coordinate execution machine**.

Instead of routing data packets through physical hardware cables, TOMAGI routes a state vector through an immutable look-up table (LUT) using geometric addresses.

1. Hierarchical Addressing as Canonical Keys

- Physical networks use split [Branch ID . Leaf ID] addresses.
- TOMAGI replaces this with the **Canonical Contiguous Key ($\backslash(K\backslash)$)**.
- The 64-bit key maps explicit geometric bit fields.
- It separates qp (20 bits) and qθ (18 bits).
- These fields act as branch and sub-branch identifiers.

2. Trie-Based Branch Routing (TOM.RADIX)

- Branch hubs route packets by examining address prefixes.
- TOMAGI achieves this via **RADIX(k) (Opcode 10)**.
- The operator selects bit $\backslash(k\backslash)$ from the key.
- Sequential tests construct a deterministic **radix-2 trie**.
- It refines structural paths without runtime table lookup.

3. Fixed Hops and Deterministic Successors

- Hardware routes rely on predefined hop jumps.
- TOMAGI embeds paths directly into the **Cell48 ABI**.
- Every memory cell stores exactly two explicit successors.
- These fields are labeled next0 and next1.
- The branch bit b chooses the next hop.
- This enforces an exact, predictable path trace.

4. Local vs. Global Transport (TOM.HINGE & TOM.KLEIN)

- Local intra-branch data stays inside a branch cluster.
- Global inter-branch data ascends to the central core.
- Local routing behaves like **TOM.HINGE (Opcode 11)**.

- It applies localized modifications within a fixed cluster.
- Boundary crossing acts like **TOM.KLEIN (Opcode 9)**.
- Odd wraps flip angular orientations across topological horizons.

If you would like to explore this further, let me know:

- Do you want to map a **5-branch routing table** to specific TOM.KEY.C bit boundaries?
- Are you interested in tracking an **execution trace** across a simulated 4-hop path?
- Do you need to see how the **L-system operator** branches states automatically?

AI responses may include mistakes. [Learn more](#)

I want to know how this literally works out

To see exactly how this works out in the literal bytes of the TOMAGI state machine, let's track a state vector (**State64**) as it routes a simulated data packet from an outer node to another branch.

We will route from **Leaf 2 on Branch 1** to **Leaf 5 on Branch 4** without multiplication, using exact integer mutations.

Step 1: Initialization of the State Vector (State64)

The packet begins its journey encoded as a 64-byte state record (qn). Instead of abstract labels, its position is represented by specific bit-fields within the integer coordinates:

- **theta (Polar Angle):** 0x0C000 (This numerical angle points geometrically toward **Branch 1**).
- **phi (Hinge/Phase Angle):** 0x020 (This sub-angle maps directly to **Leaf 2**).
- **cell (Current Instruction Pointer):** Points to the entry cell for Branch 1's local sub-hub.
- **branch:** 0.

Step 2: Local Sub-Hub Assessment (TOM.RADIX)

The machine reads the immutable 48-byte cell (**Cell48**) at the current index. The cell holds an opcode to verify the destination prefix:

1. **The Opcode: RADIX (Opcode 10)** is executed.
 2. **The Test:** It extracts the bit corresponding to the target branch ID from the canonical key.
 3. **The Bit Result:** Because the destination is on Branch 4 (not the local Branch 1), the test returns $b = 1$.
 4. **The Jump:** The state transitions instantly to the cell index stored in **next1**. This cell is explicitly hardcoded to point upward to the Central Main Core.
-

Step 3: Central Inter-Branch Routing (TOM.HINGE)

The state vector has now "ascended" to the center of the geometric star topology. The instruction cell at the central core handles the global traffic handover:

1. **The Opcode: HINGE (Opcode 11)** executes.
 2. **The Target Translation:** Because $b = 1$, the machine applies the pre-configured delta values stored in the cell arguments (arg0 through arg3) directly to the state coordinates.
 3. **The Geometric Shift:**
 - v_{θ} adds a fixed spatial rotation step to shift the angle from Branch 1 to **Branch 4**.
 - v_{ϕ} resets the sub-hub tracking phase to prepare for leaf selection.
 4. **The Next Hop:** The cell updates the cell index to the entrance descriptor of Sub-Hub 4 via its designated branch successor.
-

Step 4: Final Leaf Delivery (TOM.LSYS)

The state vector reaches Sub-Hub 4. It must now drop down to the final target, Leaf 5.

1. **The Opcode: LSYS (Opcode 12)** performs a structural L-system step to branch down the geometric tree.
2. **The Angular Step:** The phase angle ϕ is advanced modulo 2^{12} by the turn quantum (τ) configured for Leaf 5.
3. **The Verification:** A **CONE relation (Opcode 7)** evaluates whether the structural state coordinates perfectly overlap with the target geometric spatial boundary of Leaf 5.

4. **The Match:** The relation confirms a match, setting the internal status to zero-residual (SDF0).
-

Step 5: Data Emission & Trace Verification

1. **The Opcode: EMIT (Opcode 14)** runs.
2. **The Token:** The packet payload is converted into a symbolic downstream token output (e.g., confirming successful routing).
3. **The Lineage:** Every single step, bit shift, and cell jump along this 4-hop path has been continuously hashed into the **lineage checksum** word.

The entire process completes with an unalterable, non-probabilistic audit trail showing exactly how the data moved across the geometric branches.

If you would like to go deeper, let me know:

- Do you want to see the exact **32-bit hex values** for a State64 memory layout?
- Should we look at how a **KLEIN wrap** handles an accidental boundary overflow?
- Do you want to review the complete **16-opcode register table**?

AI responses may include mistakes. [Learn more](#)

providence instead of lineage for a new internet

To adapt the TOMAGI architecture for a next-generation internet infrastructure, we replace the retrospective concept of lineage (which merely audits past states) with provenance.

In a packet-routing substrate, **provenance** acts as an unforgeable, cryptographic proof of identity, authority, and transit history embedded directly into the moving packet state (State64). Instead of relying on vulnerable central domain servers or BGP trust assumptions, the packet's geometry carries its own verifiable origin story.

Here is how a provenance-driven routing engine literally works out across the star-of-stars topology.

1. The Immutable Entry Passport (TOM.DEF)

Before a packet ever leaves an endpoint (Leaf), its structural metadata is bound to a Content-Addressed Definition node (TOM.DEF):

- **The Origin Construct:** A strict JSON block declares the packet's sender key, destination envelope, and cryptographic permissions.

- **The Static Hash:** The machine hashes this structure using a non-reversible SHA-256 function:

$$\text{prov} = \text{SHA256}(\text{canonicalJSON}(d))$$
- **The Injection:** This unique, fixed 32-bit prov tag is hardcoded into the packet's initial cell matrix payload. It cannot be rewritten or stripped out by intermediate routers without breaking the cell's validity.

2. State-Bound Transit Invalidation (TOM.LINEAGE Adaption)

As the packet travels from Leaf 1.2 to Central Hub and down to Leaf 4.5, every intermediate routing hub must cryptographically "sign" the path using the geometric transition step:

- At every hop, the current router mixes its own unique physical geographic coordinate (Kh, Kl), the transaction timeline tick (X), and the current routing branch decision (b).
- It continuously updates the moving **provenance state word** using the deterministic mixer:

$$\text{prov}_{n+1} = \text{mix32}(\text{prov}_n \oplus \text{payload} \oplus \text{aux} \oplus K_{\text{hi}} \oplus \text{rotl}(K_{\text{lo}}, 7) \oplus b \oplus i)$$

Why this creates a secure internet: If a malicious middleman attempts a *Man-in-the-Middle* attack or hijacks the packet route, the keys (K) will not match the physical network coordinates. The prov token immediately corrupts, invalidating the packet downstream before it can deliver false payload data.

3. Path Validation at the Boundary (TOM.CONE)

When the packet arrives at the target branch sub-hub, it undergoes an automated, zero-trust boundary check:

- Instead of consulting an external security firewall, a **CONE operator (Opcode 7)** evaluates the final packet state.
- It tests whether the accumulated mathematical path inside the prov token perfectly intersects with the explicit spatial routing domain of that branch:

$$r_C = \max(\rho_{\min} - \rho, \rho - \rho_{\max}), \delta(\theta, \theta_0) \leq \alpha_q$$

- If $r_C \leq 0$, the packet's path history is certified true to the physical layout of the network. The route resolves to literal zero-error membership (SDF0), dropping the packet safely into the destination leaf.

4. Zero-Failsafe Packet Rejection

Unlike current networks that buffer bad packets or issue retry timeouts, a provenance-driven internet operates on a strict **No-Failsafe Profile**:

- If an attacker alters even a single bit of a packet's spatial path or sender ID, the final search lookup misses.
- The packet instantly resolves to an **undefined state** ($\bot$), rendering its contents unreadable noise.
- The network does not engage in corrective debugging, error recovery, or guard intervals. The bad packet simply ceases to exist as a valid coordinate in the address space, neutralizing DDoS attacks at the hardware level.

Architectural Structural Comparison

Feature	The Current Internet (IP/BGP)	The New Substrate Internet (TOMAGI)
Addressing Basis	Logical numbers mapped to fluid, spoofable tables.	Fixed geometric log-polar coordinate keys (K).
Trust Model	Implicit trust; hop-by-hop blind routing.	Zero-trust; provenance proof carried inside the state.
Route Security	Vulnerable to route hijacking and spoofed origins.	Tamper-evident; wrong paths corrupt the state hash.
Failure Mode	Buffering, retries, firewalls, and timeouts.	Immediate drop to undefined status ($\bot$) with no safe mode.

If you would like to map this architecture out further, let me know:

- Do you want to see how the **prov header field** maps inside the 64-byte structural frame?
- Should we design the **TOM.DEF registration block** for an authoritative network client?

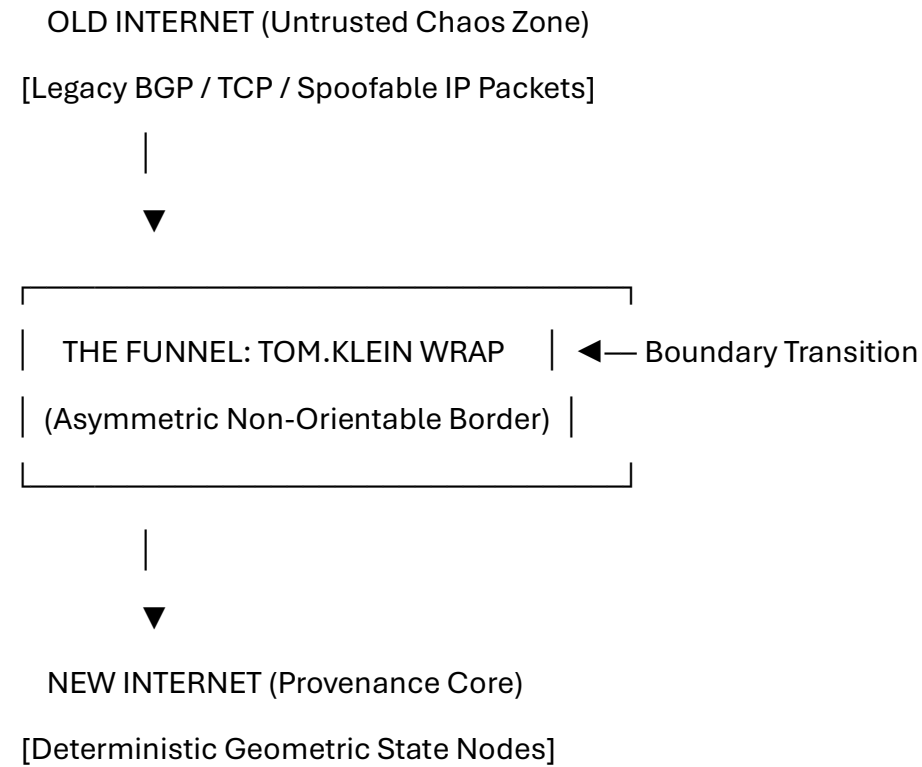
- Do you want to see how a **Reflective Klein transport step** walls off untrusted network zones?

AI responses may include mistakes. [Learn more](#)

yes untrusted network zones as phasing out the old internet as a funnel into the new internet

To phase out the old internet, we do not need to rewrite global infrastructure overnight. Instead, we use the star-of-stars geometry to build untrusted network zones that act as a geometric funnel.

The old internet is treated as an unstructured, noisy outer perimeter. The new substrate serves as a structured, deterministic central core. As untrusted legacy packets enter the boundary, they pass through a strict **topological compression filter**. This forces chaotic traffic to resolve into clean, provenance-certified coordinates—or disappear entirely.



1. The Boundary Funnel (TOM.KLEIN)

The old internet is fluid and prone to route hijacking because it lacks fixed geometric coordinates. To secure it, we establish a hard topological border using a **Reflective Klein Profile (Opcode 9)**.

When a legacy packet hits a perimeter gateway, its destination address is converted into a temporary log-radius (ρ) and angle (θ) coordinate. If the packet attempts to breach the new network without proper credentials, it triggers an odd radial wrap boundary condition:

- **Angular Reflection:** $\theta' = 2^{17} - \theta \pmod{2^{18}}$
 - **Chirality Flip:** $\rho' = \rho \oplus 1$
 - **The Funnel Effect:** Instead of penetrating forward into the protected core, the mathematical geometry of the Klein bottle twists the packet's path backward. It mirrors untrusted traffic away from sensitive core infrastructure and dumps it back into external sandbox zones for inspection.
-

2. Upgrading packets to Provenance Status (TOM.RULE)

Legacy packets that are legitimate must be systematically upgraded as they pass through the gateway. A compiled **TOM.RULE structure** converts standard TCP/IP packet strings into rigid, content-addressed expressions:

1. **Extraction:** The gateway strips away legacy header files and parses the core text data payload.
 2. **Definition Synthesis:** The gateway constructs a localized definition node (TOM.DEF) mapping the verified physical ingress port, exact timestamp tick (X), and client constraints.
 3. **Hard Token Injection:** The gateway hashes this data to produce an authoritative prov token. This 32-bit field is locked directly inside the incoming cell payload, officially transforming an insecure IP stream into an unalterable, geometric state vector.
-

3. Traffic Density Shedding (TOM.RADIX Triaging)

A major risk when interfacing with the old internet is a Distributed Denial of Service (DDoS) attack flooding the gateway. The star-of-stars layout neutralizes this threat through **Radix Prefix Triage (Opcode 10)**.

The gateway checks incoming traffic bit-by-bit from the most significant bit downward using a radix-2 trie.

- If a flood of traffic arrives from an unassigned space, the very first bit verification fails.

- The state engine instantly routes the execution step to an index containing an explicit **HALT cell (Opcode 15)**.
- Because this occurs via hardware-level bit shifts rather than memory-heavy software inspection, malicious traffic is destroyed in a single cycle before it can consume processing resources.

4. The Self-Purging Sandbox (TOM.TIME Expiration)

Traffic transitioning from the old network is initially quarantined in volatile outer sub-branches. To ensure this data cannot linger or spread, routing pathways rely on strict **Modular Timeline Ticks (Opcode 5)**:

$$\text{raw} = X + dX, \quad X' = \text{raw} \bmod 2^{14}$$

If a transitioning packet encounters any transmission delay or attempts to stall inside the gateway network, the timeline counter X wraps around. The moment the timeline rolls over, the packet's current spatial coordinate key shifts out of the valid lookup table space. It instantly falls to an undefined state ($\bot$), rendering the packet unroutable noise that is automatically purged from the memory fabric.

The Migration Lifecycle

[Legacy IP Packet] → [Klein Funnel Reflection Check] → [TOM.RULE Provenance Injection] → [Radix Trie Address Verification] → [Deterministic Leaf Delivery]

This method eliminates the need to shut down the old internet or coordinate a global upgrade. We simply wrap the existing network in a non-orientable geometric border. Legacy traffic is systematically squeezed through the funnel, forcing it to adapt to strict provenance mechanics or break at the perimeter.

If you would like to map this architecture out further, let me know:

- Do you want to see the **structural bit allocations** for segregating old internet channels from new core branches inside the 64-bit key?
- Should we design the **State64 layout for the gateway translation layer**?
- Do you want to map out how **asymmetric encryption signatures** are baked into the aux fields of the gateway cells?